

ATIVIDADE 11

Análise estática e refatoração em Java

UC: Garantia da Qualidade de Software | Projeto: ProcessadorPagamento

Resultado

As três ferramentas reduziram a zero os problemas registrados no código refatorado. O original foi compilado e analisado com as mesmas regras e versões. Foram feitas 230 verificações funcionais e de configuração; todas passaram.

1. Registro de problemas encontrados antes da refatoração

Linhas e descrições correspondem aos diagnósticos efetivos. As categorias e severidades são a classificação acadêmica aplicada para preencher a tabela do enunciado.

ID	Ferramenta	Linha	Descrição	Tipo	Severidade
1	Checkstyle	3	Import wildcard java.util.* reduz clareza e controle de dependencias	Estilo / manutenibilidade	Baixa
2	Checkstyle	5	Classe sem documentacao Javadoc	Estilo / manutenibilidade	Baixa
3	Checkstyle	7	Metodo processar sem documentacao Javadoc	Estilo / manutenibilidade	Baixa
4	Checkstyle	40	validarSenha sem documentacao Javadoc	Estilo / manutenibilidade	Baixa
5	Checkstyle	49	calcularDesconto sem documentacao Javadoc	Estilo / manutenibilidade	Baixa
6	SpotBugs	42	Comparacao de senha String com == verifica identidade, causando autenticacao incorreta	Bug	Alta
7	Semgrep	41	Senha fixa hardcoded em senhaCorreta	Segurança	Alta
8	Semgrep	42	Comparação de String usando ==	Bug	Alta

Observação: o exemplo do PDF usa "senha==senhaCorreta" e senha fixa. O SpotBugs registrou esse bug uma vez; o Semgrep registrou os dois problemas separadamente para que cada correção fique rastreável.

2. Comparação antes/depois

Métrica	Antes	Depois	Melhoria
Total de problemas (Checkstyle, 4 espaços)	5	0	100% menos
Total de problemas (SpotBugs)	1	0	100% menos
Total de problemas (Semgrep: regras locais)	2	0	100% menos
Complexidade ciclomática máxima	9	6	33% menos
Profundidade máxima de if aninhado	3	0	100% menos
Comparação de senha String com ==	1	0	100% menos
Senha fixa no código (Semgrep)	1	0	100% menos
Números mágicos (Checkstyle dedicado)	9	0	100% menos

Regra de contagem: as ocorrências do Checkstyle principal foram medidas com as regras Google e indentação ajustada para quatro espaços, como pede a atividade. Checkstyle no original: 5; na refatoração: 0. O Google padrão de dois espaços foi executado à parte e está registrado em XML. Como a exigência de indentação conflita com esse perfil padrão, a contagem padrão não é usada na comparação principal.

Validação funcional

APROVADO: 230 verificações. Foram comparadas 168 combinações de pagamentos e 40 combinações de descontos com a versão original, além de 10 limites de desconto, as três taxas, quatro entradas de senha e cinco configurações inválidas. Nenhuma senha de teste foi gravada em arquivo.

Decisão de segurança

A refatoração não substitui a senha fixa por outro segredo codificado. O cadastro deve fornecer hash PBKDF2-HMAC-SHA256 e salt aleatório em Base64. A verificação usa 600.000 iterações, 256 bits e comparação resistente a diferenças de tempo. Consulte a [OWASP Password Storage Cheat Sheet](#). Em produção, o cadastro e o armazenamento precisam integrar-se a um serviço de credenciais seguro.

Decisão de compatibilidade

O código conserva a regra original de exigir cartão com 16 caracteres inclusive para PIX para permitir a comparação comportamento a comportamento. O Javadoc identifica isso como limitação didática. O dinheiro continua sendo double porque essa é a interface proposta no exercício.

Ferramentas e evidências

Checkstyle 9.3 (mesmo jar da extensão VS Code), SpotBugs 4.10.4 e Semgrep 1.179.0 com duas regras locais incluídas no projeto. Os resultados completos XML/JSON, logs, código original compilado e o refatorado estão nas pastas do projeto.